

MEMORY MANAGEMENT

in C++

From Garbage Collectors to Smart Pointers by Sergino Bradford
GOUNOUKPEROU

Prologue	<i>The Garbage Collector — The Other Way</i>
Part 1	Stack vs. Heap — How Memory Actually Works
Part 2	The Dangers of Raw Pointers
Part 3	RAII — The Core C++ Idiom
Part 4	unique_ptr — Exclusive Ownership
Part 5	shared_ptr & weak_ptr — Shared Ownership
Part 6	Practical Patterns & Pitfalls

Language: C++17

Introduction & Course Overview

Why Memory Management Matters

Memory management is one of the defining characteristics of C++. Unlike Java, Python, Go, or C#, C++ does not have a garbage collector. The programmer is in direct, explicit control of when memory is allocated and when it is released. This is both C++'s greatest strength — giving you predictable, measurable performance — and its most dangerous trap.

Before we dive into how C++ handles memory, we need to understand the alternative — the garbage collector — because C++'s entire design philosophy is a deliberate response to it. Understanding what GCs do well, what they sacrifice, and why C++ chose a different path will make everything that follows feel inevitable rather than arbitrary.

Course Schedule

0:00 – 0:10	Introduction — Why Memory Management Matters
0:10 – 0:28	Prologue — The Garbage Collector: The Other Way
0:28 – 0:40	Part 1 — Stack vs. Heap: How Memory Actually Works
0:40 – 0:52	Part 2 — The Dangers of Raw Pointers
0:52 – 1:04	Part 3 — RAIL: The Core C++ Idiom
1:04 – 1:16	Part 4 — <code>unique_ptr</code> : Exclusive Ownership
1:16 – 1:25	Part 5 — <code>shared_ptr</code> & <code>weak_ptr</code> : Shared Ownership
1:25 – 1:30	Part 6 — Practical Patterns & Pitfalls

Prerequisites

- Comfortable with C basics (pointers, structs, functions)
- Familiar with C++ classes and basic OOP
- Has encountered a segfault or memory leak at least once (almost guaranteed at Epitech)



The Garbage Collector

The Other Way — and Why C++ Chose Differently

The Problem That Created the Garbage Collector

To understand why garbage collectors were invented, we need to go back to the 1950s. John McCarthy, while designing LISP in 1959, faced a simple but brutal problem: he needed to allocate and deallocate memory for symbolic expressions at runtime, and he realized that forcing the programmer to track every allocation manually was error-prone, tedious, and fundamentally incompatible with the high-level abstractions he wanted to build.

His solution was the first garbage collector: a runtime system that periodically scanned memory, identified objects that were no longer reachable from the program, and automatically freed them. The programmer could allocate freely — the runtime handled deallocation. The idea was revolutionary, and over the next 60 years it became the dominant memory management strategy in high-level languages: Java (1995), Python (1991), C# (2000), Go (2009), JavaScript (1995) all use GC.

The core promise of a GC is compelling: you, the developer, never write 'free' or 'delete'. You just create objects, use them, and when you stop referencing them, they disappear. Memory bugs like use-after-free and double-free become structurally impossible.

How a Garbage Collector Works

Modern garbage collectors are sophisticated pieces of engineering. Understanding how they work explains both their strengths and their costs.

Step 1 — Allocation

In a GC-managed language, all objects live in a special region called the managed heap. Allocation is typically very fast — often just a pointer bump into a pre-allocated arena, faster than C's malloc in the common case.

Step 2 — Reachability Tracking

The GC maintains the concept of 'roots': references held in CPU registers, stack frames, and global variables. An object is considered 'live' if it is reachable from any root, directly or through a chain of references. An object is 'garbage' if no path from any root leads to it.

```
// Java — the GC handles all of this silently
public void process() {
    MyObject obj = new MyObject(); // allocated on managed heap
    obj.doWork();
    // obj goes out of scope here.
    // Is it freed? Not immediately — the GC decides when.
    // The programmer never knows, never cares.
}

// No delete, no free, no destructor call.
```

```
// If another reference to obj exists elsewhere, it stays alive.  
// When all references are gone, it becomes eligible for collection.
```

Step 3 — Collection

The collector runs periodically (or when memory pressure triggers it) and reclaims garbage. The most common algorithm in modern runtimes is the generational mark-and-sweep collector with compaction:

- Mark phase: starting from roots, traverse all live object references, marking each object as reachable.
- Sweep phase: scan the heap for unmarked (unreachable) objects and reclaim their memory.
- Compaction: optionally move live objects together to eliminate heap fragmentation.
- Generational hypothesis: most objects die young. Short-lived objects live in a 'young generation' collected frequently and cheaply; long-lived objects are promoted to an 'old generation' collected rarely.

What the GC Gets Right

Garbage collection genuinely eliminates whole categories of bugs. This is not a small thing — it is the reason most modern languages adopted it:

Bug type	GC eliminates it?
Use-after-free	✓ Yes — GC never frees an object while a reference exists
Double-free	✓ Yes — the programmer never calls free at all
Memory leak	✓ Mostly yes — unreachable objects are collected automatically
Dangling pointer	✓ Yes — references are always valid or null, never dangling
Buffer overflow	⚠ Partially — bounds-checked arrays prevent most, not all cases

The Costs of Garbage Collection

The GC's safety guarantees come with real, measurable trade-offs. These costs are acceptable — even invisible — in most applications. But in the domains where C++ dominates, they are decisive.

Cost 1 — Stop-The-World Pauses

Most GC algorithms need to pause all application threads while they run the mark phase. This ensures the object graph doesn't change while it's being traversed. In a Java or C# application, you can observe GC pauses of 10ms to hundreds of milliseconds. Modern concurrent GCs (like Java's ZGC or G1) have reduced this dramatically, but pauses cannot be fully eliminated without enormous engineering complexity.

For a web server responding to HTTP requests, a 50ms pause is annoying. For a real-time game rendering at 60fps (where each frame has a 16ms budget), a 50ms GC pause means three dropped frames and a visible stutter. For a financial trading system executing orders in microseconds, a 50ms pause could mean millions of dollars lost. For embedded systems running on microcontrollers, a GC runtime may not even fit in available memory.

Cost 2 — Unpredictable Latency

Beyond the pauses themselves, GC introduces non-determinism into an application's timing profile. You cannot predict when collection will run, how long it will take, or what its impact will be on a given request. This makes it very difficult to provide latency guarantees — which is a hard requirement in many systems.

Cost 3 — Memory Overhead

A GC runtime needs headroom to operate efficiently. It typically requires 2–5x more live memory than the actual working set of the application — the extra space lets the allocator batch collections and reduce fragmentation. A Java process that uses 200MB of actual data might need 600MB–1GB of heap to run without thrashing the GC. This is a real constraint on memory-limited systems.

Cost 4 — No Deterministic Destruction

This is perhaps the most profound cost from a software design perspective. In a GC language, you never know exactly when an object will be destroyed. The GC decides. This means you cannot reliably tie the release of non-memory resources — file handles, network connections, database transactions, mutex locks — to object destruction. You are forced to use explicit 'close()' or 'dispose()' calls, try-finally blocks, or try-with-resources patterns. The GC handles memory; everything else is your problem.

```
// Java – you must explicitly close resources:
// The GC will eventually call finalize(), but 'eventually' is not good enough
// for a file handle that needs closing NOW.
try (FileWriter fw = new FileWriter("log.txt")) {
    fw.write("data");
} // FileWriter.close() called here – by the try-with-resources, NOT the GC

// Python – same problem, same solution:
with open('log.txt', 'w') as f:
    f.write('data')
# f.close() called here by the 'with' statement context manager

// C++ – the destructor handles this AUTOMATICALLY and DETERMINISTICALLY:
{
    std::ofstream f("log.txt");
    f << "data";
} // f's destructor runs HERE, closing the file – always, no exceptions
```

 The 'try-with-resources' in Java and 'with' in Python are manual workarounds for the problem that GC languages don't have deterministic destruction. C++ solves this problem at the language level with destructors and RAII — which we'll cover in depth in Part 3.

Why C++ Chose Its Approach

C++ was designed by Bjarne Stroustrup starting in 1979 as 'C with Classes'. Its design goals were explicit and uncompromising: it had to be at least as fast as C, it had to provide zero-overhead abstractions (you only pay for what you use), and it had to be usable in the same domains as C — operating systems, device drivers, game engines, embedded systems, real-time control systems.

A garbage collector was considered and rejected for three reasons that remain just as valid today:

- Zero-overhead principle: a GC runtime imposes memory and CPU overhead on every program, including those that don't need it. C++ refuses to make you pay for features you don't use.
- Deterministic destruction: C++ programs often manage non-memory resources. The ability to tie arbitrary cleanup to scope exit — via destructors — is more powerful and general than what a GC provides.
- Real-time constraints: GC pauses make a language unsuitable for hard real-time systems. C++ must be deployable in aircraft control systems, medical devices, and game engines where latency guarantees are non-negotiable.

This does not mean C++ considers memory safety unimportant. On the contrary — the language evolved a powerful, zero-cost answer to the GC's safety guarantees: the RAII idiom and smart pointers. Instead of a runtime that cleans up after you, C++ gives you compile-time constructs that make it structurally impossible to forget to clean up. The safety is enforced by the type system at compile time, not by a runtime at execution time.

The Landscape Today

The tradeoffs between GC and manual management are not resolved — they are situational. The right tool depends on your requirements:

Requirement	GC Language (Java, Python...)	C++ (RAII + Smart Ptrs)
Developer productivity	✓ High — no memory bugs	⚠ Requires discipline
Predictable latency	⚠ GC pauses are a risk	✓ Fully deterministic
Memory efficiency	⚠ 2–5x overhead typical	✓ Minimal overhead
Real-time / embedded	✗ Usually unsuitable	✓ Standard choice
Non-memory resources	⚠ Manual close() required	✓ Destructor handles all
Safety against mem bugs	✓ Structural guarantees	✓ With smart pointers

 *Rust takes a third approach: a compile-time borrow checker that provides GC-level safety guarantees with C++-level performance — no runtime overhead, no pauses. It is the most promising modern answer to this problem, but C++ remains dominant in its core domains due to its ecosystem and legacy.*

With this context established — what GC is, what it costs, and why C++ made different choices — we can now understand C++'s memory model not as a burden, but as a deliberate, powerful design. Let's start from the foundation.



Stack vs. Heap

How Memory Actually Works

The Two Memory Regions You Use Every Day

Every running program has (at minimum) two regions of memory it actively uses: the stack and the heap. Unlike a GC-managed runtime where both are abstracted away, in C++ you interact with them directly — so understanding the difference is the foundation of everything else.

The Stack

The stack is a region of memory that grows and shrinks automatically as functions are called and return. Every time you call a function, a 'stack frame' is pushed: it contains the function's local variables, its parameters, and the return address. When the function returns, the frame is popped and that memory is instantly reclaimed.

```
void foo() {  
    int x = 10;           // lives on the stack  
    double y = 3.14;      // lives on the stack  
    char buf[256];        // 256 bytes on the stack  
    // When foo() returns, x, y, and buf are GONE automatically  
}
```

Stack allocation is essentially free — it's just a single register increment. Stack memory is also cache-friendly and automatically managed. The downside: the stack is small (typically 1–8 MB) and its lifetime is tied strictly to the enclosing scope. You cannot return a pointer to a local variable.

✗ BAD — Memory hazard

```
int* dangerousFunction() {  
    int localVar = 42;  
    return &localVar; // BUG: returning address of stack variable!  
}                     // localVar is destroyed when function returns  
                     // The pointer now points to garbage memory
```

The Heap

The heap (also called the free store in C++) is a large, manually managed region of memory. You request blocks of it with 'new' and release them with 'delete'. Heap-allocated memory lives until you explicitly free it — it outlives the function that created it. This is equivalent to what a GC manages automatically in other languages, except here, you are the garbage collector.

```

int* createValue() {
    int* p = new int(42); // allocates on the heap
    return p;             // safe: heap memory outlives the function
}

int main() {
    int* val = createValue();
    std::cout << *val;    // 42 — still valid
    delete val;           // must manually free!
    val = nullptr;        // good practice after delete
}

```

Stack vs. Heap at a Glance

Property	Stack	Heap
Allocation	Automatic (scope entry)	Manual (new / delete)
Deallocation	Automatic (scope exit)	Manual — your responsibility
Speed	Extremely fast	Slower (system call, fragmentation)
Size	Small (~1–8 MB)	Large (limited by RAM)
Lifetime	Tied to scope	Until explicitly freed
GC analogy	N/A — always deterministic	This is what GCs manage for you

💡 *Rule of thumb: prefer the stack whenever possible. Only use the heap when you need dynamic lifetime, large allocations, or polymorphism.*



The Dangers of Raw Pointers

Every bug you will encounter

Raw Pointers Are a Tool, Not a Design

Raw pointers (`int*`, `MyClass*`) are the low-level mechanism C++ uses for indirect memory access. They are powerful and necessary at the machine level, but using them as your primary memory management strategy in application code leads to a predictable set of painful bugs. These are the exact bugs that garbage-collected languages eliminate structurally — but as we've seen, GC comes with costs. The C++ answer, coming up in Part 3, eliminates these bugs without those costs.

Bug #1: Memory Leak

A memory leak occurs when you allocate heap memory and never free it. The program keeps consuming memory until the OS kills it or performance degrades catastrophically. This is the most common bug in beginner C++ code.

✗ BAD — Memory hazard

```
void processRequest() {
    MyData* data = new MyData();
    data->load();

    if (data->isEmpty()) {
        return;                // BUG: early return — delete is never called!
    }

    data->process();
    delete data;                // Only reached in the happy path
}
// Called thousands of times per second → server crashes after hours
```

💡 GC languages make this bug impossible: the collector reclaims unreachable objects. In C++, RAI and smart pointers eliminate it at zero runtime cost — as we'll see shortly.

Bug #2: Dangling Pointer (Use-After-Free)

A dangling pointer points to memory that has already been freed. Reading from it produces garbage values. Writing to it silently corrupts other data — or, in security contexts, enables arbitrary code execution. This is the most dangerous class of memory bug and one of the most exploited in CVEs.

✗ BAD — Memory hazard

```
int* p = new int(100);
delete p;                // memory is freed
```

```
std::cout << *p;           // BUG: use-after-free – undefined behavior!
*p = 999;                  // BUG: writing to freed memory – silent corruption

// Even more subtle – another common case:
int* getPointer() {
    int x = 42;
    return &x;              // BUG: x is on the stack – gone after return!
}
```

Bug #3: Double Free

Calling delete on the same pointer twice corrupts the heap allocator's internal data structures. This leads to crashes, undefined behavior, or security vulnerabilities.

✗ BAD – Memory hazard

```
int* p = new int(10);
int* q = p;           // q points to the same memory

delete p;              // first delete – OK
delete q;              // BUG: double free – heap corruption!

// Also common across class boundaries:
MyClass a;
MyClass b = a;         // if copy constructor just copies the pointer...
// Both a and b try to delete the same memory on destruction!
```

Bug #4: Buffer Overflow

Writing past the end of an allocated buffer corrupts adjacent memory. This is the source of countless security vulnerabilities (stack smashing, heap overflow).

✗ BAD – Memory hazard

```
char* buf = new char[8];
strcpy(buf, "Hello, World!"); // BUG: 'Hello, World!' is 14 bytes
                               // writing 6 bytes past the allocation!

delete[] buf;
```

The Root Cause: Ownership is Implicit

All four bugs above share the same root cause: with raw pointers, the question of 'who owns this memory and is responsible for freeing it?' is answered only in comments and convention — never enforced by the compiler. A GC answers this question by saying 'no one owns it; the runtime does.' Modern C++ answers it differently: 'the type system owns it,' which we explore next.



RAII

Resource Acquisition Is Initialization

The Most Important C++ Idiom

RAII (Resource Acquisition Is Initialization) is the core idiom that makes safe C++ code possible — and it is C++'s direct answer to what a garbage collector does. The idea is elegant: tie the lifetime of a resource (memory, file handle, mutex lock, network socket) to the lifetime of a stack-allocated object. When the object is destroyed — which the compiler guarantees at scope exit — the resource is automatically released.

A GC separates allocation from deallocation in time and hands deallocation to a background runtime. RAII keeps them together in the same object, ensuring they always happen in lockstep, with zero runtime overhead and fully deterministic timing.

RAII in Action: Writing It Manually

Let's build a simple RAII wrapper for a heap-allocated array, to see exactly how it works before we use the standard library versions:

✓ GOOD – Safe memory handling

```
// A simple RAII wrapper for a heap-allocated array
class ManagedBuffer {
    char* data_;
    size_t size_;
public:
    // Constructor: ACQUIRES the resource
    explicit ManagedBuffer(size_t size)
        : data_(new char[size]), size_(size) {}

    // Destructor: RELEASES the resource – guaranteed to run on scope exit
    ~ManagedBuffer() {
        delete[] data_;
    }

    // Disable copy to prevent double-free
    ManagedBuffer(const ManagedBuffer&) = delete;
    ManagedBuffer& operator=(const ManagedBuffer&) = delete;

    char* data() { return data_; }
    size_t size() const { return size_; }
};

void processRequest() {
    ManagedBuffer buf(1024);
    // ... use buf.data() ...

    if (someCondition)
        return; // SAFE: buf's destructor runs – memory freed!
```

```
    // SAFE: memory freed here too, no matter which path we take
}
```

Why RAI Is Bulletproof — and Better Than GC for Resources

The C++ standard guarantees that destructors run on scope exit — whether the scope exits normally, via early return, or via an exception. This makes RAI exception-safe by construction. And crucially, it works for ALL resources, not just memory:

✗ BAD — Memory hazard

```
// Manual management: every exit path must call cleanup
void riskyOperation() {
    MyResource* r = new MyResource();
    try {
        r->doSomething();    // might throw
        delete r;            // skipped if doSomething throws!
    } catch (...) {
        delete r;            // must remember to delete in EVERY catch
        throw;
    }
}
```

✓ GOOD — Safe memory handling

```
// RAI: destructor handles every exit path automatically
void safeOperation() {
    ManagedResource r;    // acquires in constructor
    r.doSomething();       // might throw — it doesn't matter
    // r's destructor runs here whether we threw or not
}
```

💡 *This is the key advantage over GC for non-memory resources: a file handle closed in a destructor is closed the moment the object goes out of scope, not 'whenever the GC feels like it.' RAI gives you the GC's safety promise plus deterministic timing plus zero overhead.*

RAI is Already Everywhere in C++

- `std::vector` — frees its heap buffer on destruction
- `std::string` — frees its character buffer on destruction
- `std::fstream` — closes the file on destruction
- `std::lock_guard` — releases the mutex on destruction
- `std::unique_ptr` — deletes the pointed-to object on destruction

The standard library is built entirely on RAI. Smart pointers are simply the RAI solution applied specifically to heap memory ownership.



unique_ptr

Exclusive Ownership — One Owner, One Lifetime

The Concept: Single Owner

`std::unique_ptr<T>` is a RAII wrapper that expresses exclusive ownership of a heap-allocated object. At any point in time, exactly one `unique_ptr` owns the object. When that `unique_ptr` is destroyed, the object is deleted. There is no ambiguity about who is responsible — the owner is the `unique_ptr`. This is C++'s zero-cost equivalent of a GC: the ownership is tracked by the type system at compile time, not at runtime.

Creating and Using `unique_ptr`

✓ GOOD — Safe memory handling

```
#include <memory>

// PREFERRED: use make_unique (C++14) — exception-safe
auto ptr = std::make_unique<MyClass>(arg1, arg2);

// Access like a raw pointer
ptr->method();
(*ptr).method();

// Get the raw pointer (without releasing ownership)
MyClass* raw = ptr.get();

// unique_ptr automatically deletes when it goes out of scope
// No need to call delete — ever.

// Array version:
auto buffer = std::make_unique<char[]>(1024);
buffer[0] = 'H';
```

Ownership Transfer with `std::move`

`unique_ptr` cannot be copied — that would create two owners, which its semantics forbid. But ownership can be transferred (moved) to another `unique_ptr`. After the move, the original is null. This is ownership transfer: explicit, visible in the code, checked by the compiler.

✓ GOOD — Safe memory handling

```
auto a = std::make_unique<MyClass>();

// auto b = a;           // ERROR: copy is deleted — the compiler protects you

auto b = std::move(a); // OK: ownership transferred to b
```

```

// a is now nullptr

// Passing ownership to a function:
void takeOwnership(std::unique_ptr<MyClass> obj) {
    // obj owns the resource here
} // obj destroyed here → MyClass deleted

takeOwnership(std::move(b)); // b is null after this call

```

unique_ptr in Real Code: Factory Functions

unique_ptr is the ideal return type for factory functions. It makes ownership crystal clear: the caller receives exclusive ownership and the compiler enforces it. This is something a GC language simply cannot express — all references in Java or Python are equivalent, and nothing in the type system tells you who 'owns' an object.

✓ GOOD – Safe memory handling

```

// Factory: creates and returns ownership to caller
std::unique_ptr<Shape> createShape(ShapeType type) {
    if (type == ShapeType::Circle)
        return std::make_unique<Circle>(5.0);
    if (type == ShapeType::Rectangle)
        return std::make_unique<Rectangle>(4.0, 6.0);
    return nullptr;
}

auto shape = createShape(ShapeType::Circle);
if (shape) {
    std::cout << shape->area();
}
// shape deleted automatically – no delete needed

// Polymorphism works perfectly with unique_ptr:
std::vector<std::unique_ptr<Shape>> shapes;
shapes.push_back(std::make_unique<Circle>(3.0));
shapes.push_back(std::make_unique<Rectangle>(2.0, 5.0));
// All shapes auto-deleted when the vector is destroyed

```

💡 *Default to unique_ptr. If you think you need shared_ptr, ask yourself first: can I redesign so one owner is clear? unique_ptr is faster and has clearer semantics.*



shared_ptr & weak_ptr

Shared Ownership — When Multiple Owners Are Needed

shared_ptr: Reference-Counted Ownership

`std::shared_ptr<T>` allows multiple pointers to share ownership of the same object. Internally, it maintains a reference count — a counter of how many `shared_ptr`s point to the object. When the last `shared_ptr` is destroyed, the reference count drops to zero and the object is deleted. If this sounds familiar, it should: this is conceptually very close to how a tracing GC works, but scoped to a single object and counted explicitly rather than traced from roots.

✓ GOOD — Safe memory handling

```
auto a = std::make_shared<MyClass>(); // ref count: 1
{
    auto b = a;                        // ref count: 2
    auto c = a;                        // ref count: 3
    std::cout << a.use_count();        // prints 3
}
// b and c destroyed → ref count: 1
// When a goes out of scope → ref count: 0 → MyClass deleted
```

⚠ *shared_ptr is more expensive than unique_ptr: it allocates a control block and uses atomic operations for thread-safe reference counting. Do not use shared_ptr by default just because it 'seems safer.' Prefer unique_ptr wherever a single owner is identifiable.*

weak_ptr: Observing Without Owning

`std::weak_ptr<T>` is a non-owning reference to an object managed by `shared_ptr`. It does not increment the reference count. Before using a `weak_ptr`, you must 'lock' it — which gives you a temporary `shared_ptr` if the object still exists, or an empty `shared_ptr` if it has been deleted.

The Circular Reference Problem — and the GC Analogy

Tracing GCs handle circular references effortlessly — if two objects reference each other but neither is reachable from any root, both are collected. Reference counting (`shared_ptr`) cannot do this: cycles prevent the count from ever reaching zero, causing a leak. This is one genuine advantage a tracing GC has over reference counting. The C++ solution is `weak_ptr`:

✗ BAD — Memory hazard

```
struct Node {
    std::shared_ptr<Node> next; // owns next
    std::shared_ptr<Node> prev; // owns previous — CYCLE!
};
// ref counts never reach 0 — memory leaked forever
```

✓ GOOD – Safe memory handling

```
struct Node {
    std::shared_ptr<Node> next;    // owns next: forward link
    std::weak_ptr<Node> prev;     // observes previous – no ownership
};

// Using a weak_ptr safely:
if (auto parent = node->prev.lock()) { // returns shared_ptr or empty
    parent->doSomething();             // safe: parent is alive here
}
// parent shared_ptr destroyed here – ref count restored
```

Choosing the Right Smart Pointer

Type	Use when...	Overhead vs. GC
unique_ptr	One clear owner (default choice)	Zero — no runtime cost at all
shared_ptr	Multiple owners (graphs, caches)	Atomic ref count — small but real
weak_ptr	Observer, back-pointer, break cycles	Lock() call — negligible
Raw pointer	Non-owning reference / legacy API	Zero — but gives up all safety



Practical Patterns & Pitfalls

Putting it all together

The Rule of Five (and Zero)

If your class manages a resource (owns memory, a file handle, a socket), you must define all five special member functions: the destructor, copy constructor, copy assignment operator, move constructor, and move assignment operator. The Rule of Zero is the flip side: if your class only contains value types and smart pointers, define none of these — let the compiler generate them all. This is the ideal that smart pointers enable.

✓ GOOD – Safe memory handling

```
// Rule of Zero: no manual resource management needed
class Config {
    std::string name_;
    std::vector<int> values_;
    std::unique_ptr<Logger> logger_;
public:
    Config(std::string name, std::unique_ptr<Logger> log)
        : name_(std::move(name)), logger_(std::move(log)) {}
    // No destructor, no copy/move defined.
    // unique_ptr handles the Logger lifetime. The class is automatically move-
    only.
};
```

Common Pitfall: Mixing Ownership Styles

✗ BAD – Memory hazard

```
MyClass* raw = new MyClass();
std::shared_ptr<MyClass> sp(raw); // sp now owns the object
delete raw; // BUG: double delete – sp will also delete it!

// Also wrong: two independent shared_ptrs from same raw pointer
MyClass* raw2 = new MyClass();
std::shared_ptr<MyClass> sp1(raw2);
std::shared_ptr<MyClass> sp2(raw2); // BUG: separate control blocks!
```

✓ GOOD – Safe memory handling

```
// Always use make_unique / make_shared – never construct from raw pointer
auto sp = std::make_shared<MyClass>();
auto sp2 = sp; // safe: shares the same control block
```

Debugging Memory Issues: Your Tools

- Valgrind — detects leaks, use-after-free, double-frees at runtime. Run with: `valgrind --leak-check=full ./your_program`
- AddressSanitizer (ASan) — compile with `-fsanitize=address` for fast runtime detection.
- UBSan — compile with `-fsanitize=undefined` to catch undefined behavior at runtime.
- clang-tidy — static analysis that warns about raw pointer usage and ownership issues before you run the code.

💡 *Add `'-fsanitize=address,undefined'` to your Makefile during development. It will catch bugs that would otherwise take hours to debug manually.*

Conclusion & The Full Picture

The Full Arc of This Course

We started by understanding why garbage collectors were invented — to eliminate an entire class of memory bugs by making allocation and deallocation automatic. We then understood their costs: pauses, non-determinism, memory overhead, and the inability to manage non-memory resources deterministically.

C++ answered these challenges not by ignoring safety, but by embedding safety into the type system. RAII ties resource lifetime to object lifetime. Smart pointers express ownership explicitly, making the question of 'who frees this?' answerable by the compiler rather than by convention. The result is code that is as safe as GC-managed code in practice, but with deterministic timing, zero runtime overhead, and the ability to manage any resource — not just memory.

Decision Flowchart

1. Can this live on the stack? (small, scope-bound) → yes: use the stack.
2. No? Does it have one clear owner? → use `unique_ptr`.
3. Multiple genuine owners needed? → use `shared_ptr`.
4. Need to observe without owning, or break a cycle? → use `weak_ptr`.
5. Only use raw pointers for non-owning references to guaranteed-live objects.

Summary Table

Concept	What it solves	Key rule / GC parallel
GC	Memory bugs in other langs	Trades safety for pauses & non-determinism
Stack	Scope-managed locals	Prefer when size and lifetime allow
RAII	Exception-safe cleanup	GC's safety + deterministic timing + zero cost
<code>unique_ptr</code>	Leaks & ownership ambiguity	One owner — transfer with <code>std::move</code>
<code>shared_ptr</code>	Shared ownership	Like a local GC — use <code>make_shared</code>
<code>weak_ptr</code>	Cycles & observers	GC handles cycles; <code>weak_ptr</code> is our answer
Rule of Zero	Boilerplate in resource classes	Let smart ptrs handle it; define nothing

Going Further

- cppreference.com — definitive reference for `unique_ptr`, `shared_ptr`, `weak_ptr`
- Scott Meyers, *Effective Modern C++* — Items 18–22 cover smart pointers exhaustively
- 'Back to Basics: Smart Pointers' — Herb Sutter's CppCon talks (free on YouTube)
- Rust — if you want to see how ownership-based memory safety looks at the language level, without GC

— *End of Course* —